

1-Bit Magnitude Comparator :-

Truth table :

A	B	$A > B$	$A = B$	$A < B$
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

k-mappings :-

$A > B$

A \ B	0	1
0		
1	1	

$$(A > B) = A\bar{B}$$

$A = B$

A \ B	0	1
0	1	
1		1

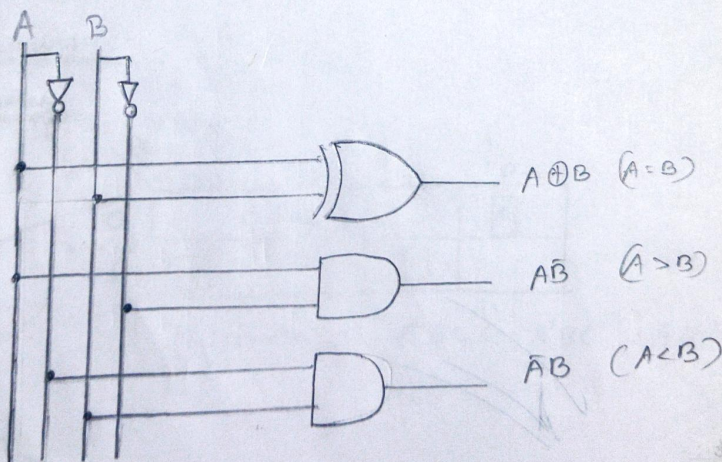
$$(A = B) = A \oplus B$$

$A < B$

A \ B	0	1
0		1
1		

$$(A < B) = \bar{A}B$$

Logical diagram :-



Ex. No: 07
16-09-25

Implementation of 1-Bit and 2-Bit Magnitude comparator using Gates

Aim:

* To design and implement 1-bit and 2-bit magnitude comparator using basic gates.

Requirements :-

* Digital trainer kit, ICs, wires, and logic gates.

Theory:

* The comparison of two numbers is an operator that determine one number is greater than, less than or equal to the other numbers.

* A magnitude comparator is a combinational circuit that compares two numbers, A and B and determine their relative magnitude.

* The outcome of the comparator is specified by three binary values that indicate whether $A > B$, $A = B$ (or) $A < B$.

* The equality of two numbers A and B is displayed in a combinational circuit designated by the symbol $(A=B)$ and this indicates A greater than B, then inspect the relative magnitude of pairs of significant digits starting from most significant position. A is 0 and that of B is 0.

1-Bit magnitude comparator :-

* It compares the two number on input variable which has only one bit each.

* Input A
B

* Output : $A < B$, $A = B$, $A > B$

2 Bit - Magnitude Comparator :-

Truth Table :-

A ₁	A ₀	B ₁	B ₀	A > B	A = B	A < B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

K-mappings :-

A > B

A ₁ A ₀ \ B ₁ B ₀	00	01	11	10
00				
01	1			
11	1	1		1
10	1	1		

$$(A > B) = A_0 \bar{B}_0 \bar{B}_1 + A_1 \bar{B}_1 + A_1 A_0 \bar{B}_0$$

A < B

A ₁ A ₀ \ B ₁ B ₀	00	01	11	10
00		1	1	1
01			1	1
11				
10				1

$$(A < B) = \bar{A}_1 \bar{A}_0 B_0 + \bar{A}_0 B_1 + \bar{A}_1 B_1$$

A = B

A ₁ A ₀ \ B ₁ B ₀	00	01	11	10
00	1			
01		1		
11			1	
10				1

$$(A = B) = (A_0 \odot B_0) (A_1 \odot B_1)$$

2-Bit magnitude Comparator:

* It compares the two numbers or input variables which has 2-bit each.

* Input $A \rightarrow A_0, A_1$

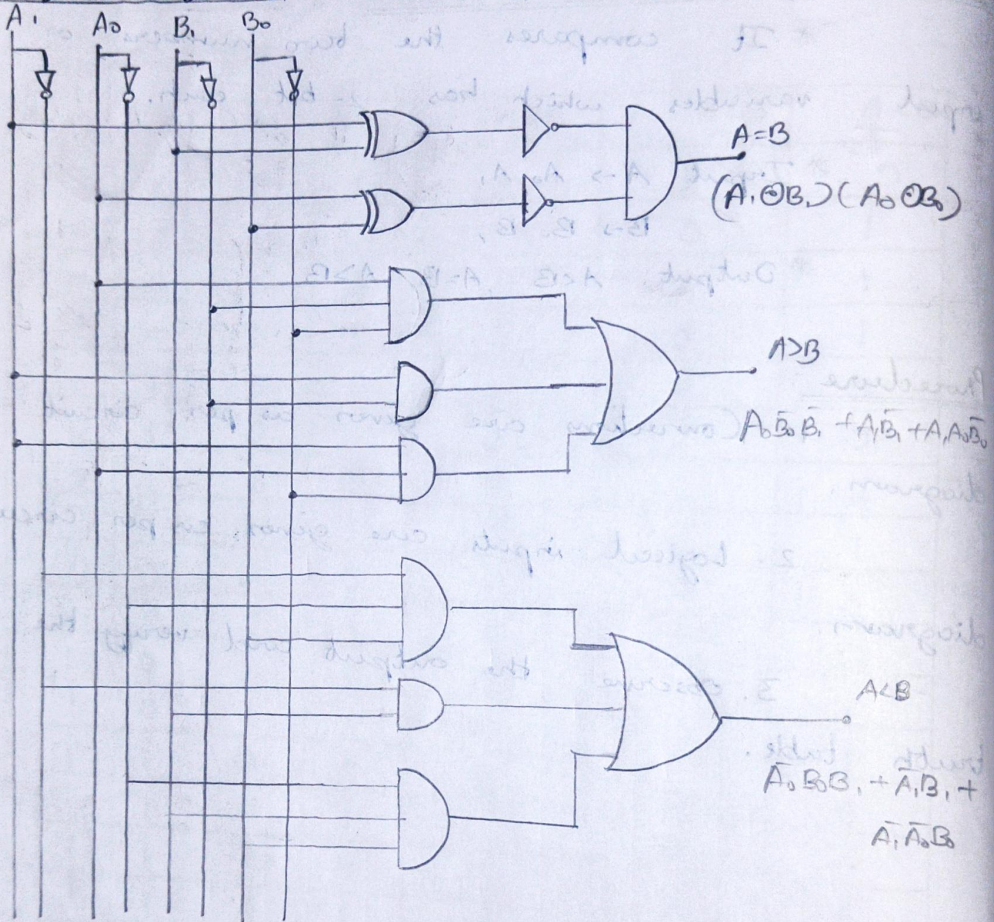
$B \rightarrow B_0, B_1$,

* Output: $A < B, A = B, A > B$

Procedure:

1. Connections are given as per circuit diagram.
2. Logical inputs are given as per circuit diagram.
3. Observe the output and verify the truth table.

Logical diagram:

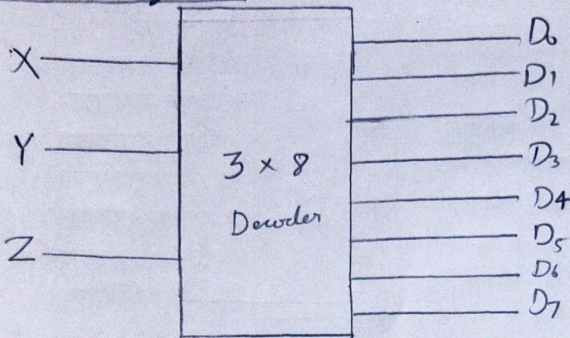


Result:-

* Thus the 1-bit and 2-bit magnitude comparators are designed and verified using truth table.

3 x 8 Decoder :-

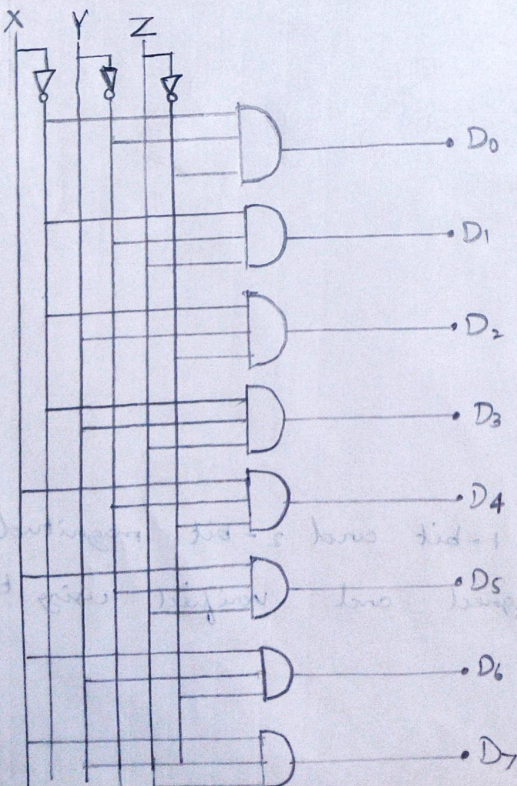
Block diagram :-



Truth table :-

X	Y	Z	D ₀	D ₁	D ₂	D ₃	D ₄	D ₅	D ₆	D ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Logical circuit



Implementation of Decoder using Logic Gates

Aim :-

* To implement 3×8 decoder using basic logic gate

Requirements :-

* Digital Trainer kit, logic gates (AND, OR, NOT), and wires

Procedure :-

3 to 8 Decoder :-

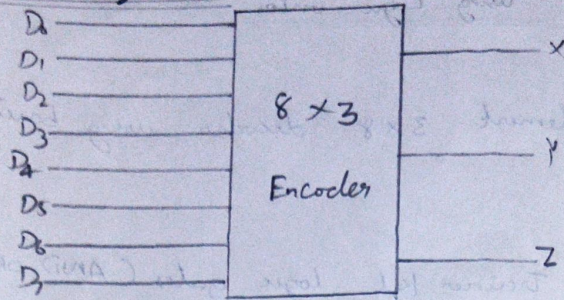
1. Use AND, OR and NOT gate.
2. Give connections as per the logic circuit.
3. There are 3 inputs and 8 outputs.
4. $\{x, y, z\}$: inputs, $\{D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7\}$: outputs
5. The outputs $\{D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7\}$ will be set as '1' when corresponding binary number is given as input.

Result :-

* Thus the 3×8 decoder is implemented using AND, OR & NOT.

Octal to Binary Encoder :-

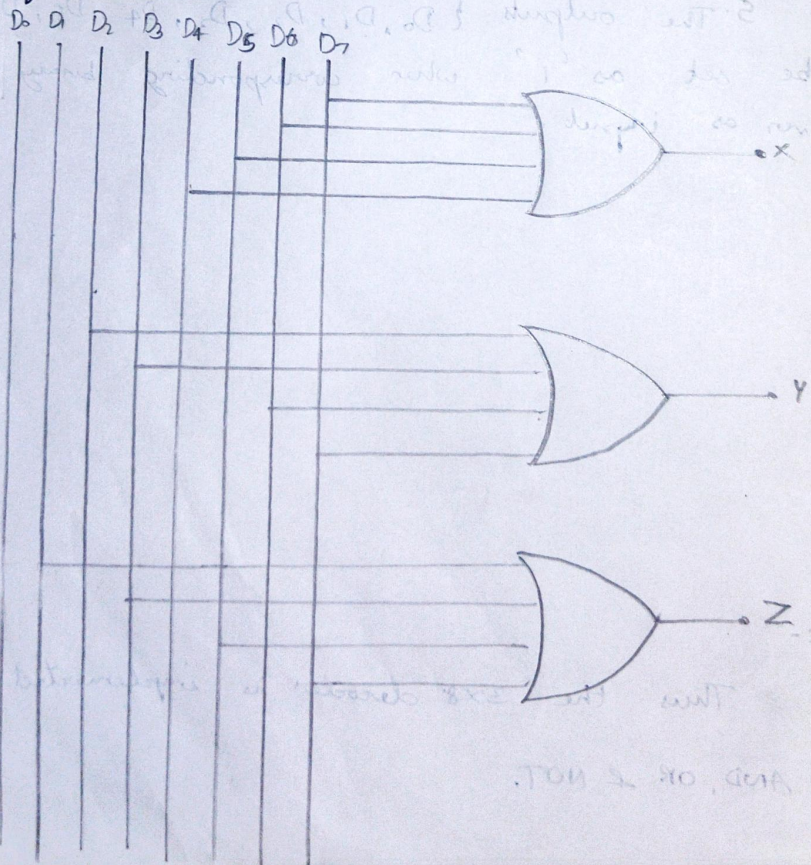
Block Diagram :-



Truth table :-

D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	X	Y	Z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Logical circuit



Implementation of Encoder using Logic gates

Aim:

* To implement octal to binary encoder using logic gates.

Requirements:

* Digital Trainer kit, Logic gates (AND, OR, NOT), and wires.

Procedure:-

1. **USE** AND, OR and NOT gates
2. Give connections as per logic circuit diagram.
3. There are 8 inputs and 3 outputs.
4. $\{D_0, D_1, D_2, D_3, D_4, D_5, D_6, D_7\}$ be inputs and $\{X, Y, Z\}$ be output.
5. When ' E_i ' is 1 where $i=0$ to 7, corresponding binary number for i will be the output in X, Y, Z .

Result:

* Thus the octal to binary encoder is implemented using AND, OR & NOT gate.